Projektuppgift

Introduktion till webbutveckling med HTML, CSS och JavaScript, DT224G

Webbplats till bageriet BAKA
Med HTML, CSS och JavaScript

Maria Westin



Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Maria Westin, mawe2607@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: HT, 2026

Sammanfattning

Som projektarbete i kursen *Introduktion till webbutveckling med HTML, CSS och JavaScript* har en webbplats till det fiktiva bageriet BAKA utvecklats.

Företaget behöver en webbplats för att potentiella kunder ska kunna hitta information om bageriet, se sortimentet, öppettider och kontaktuppgifter.

Arbetet började med planering genom wireframes och mockups. Sedan startade utvecklingen i Visual Studio Code. Arbetet versionhanterades i Github och publicerades sedan på Netlify.

Webbplatsen presenterar företaget och dess sortiment samt kontaktuppgifter. Den består av fyra HTML-sidor i en responsiv design som anpassar innehållet till skärmstorleken.

CSS har använts för att styra webbplatsens utseende och JavaScript har använts för att skapa en hamburgermeny, läsa in artiklar från en extern jsonfil, samt för att validera kontaktformuläret.

Slutsatsen är att de uppsatta målen med projektet har uppfyllts och webbplatsen fungerar som tänkt i testerna.

Innehållsförteckning

Sammanfattning iii

1 Inledning 1

2 Teori 2

3 Metod 4

4 Konstruktion 6
4.1 Planering 6
4.2 HTML & CSS 7
4.3 JavaScript 9

5 Resultat 10
5.1 Test 12

6 Slutsatser 13

Källförteckning 14

Inledning

Min dotter drömmer om att jobba i ett bageri när hon blir stor och vi brukar ibland fantisera om att starta ett bageri tillsammans. Webbplatsen ska därför presentera vårt fantasibageri som jag döpt till BAKA.

Bageriet BAKA erbjuder bröd och bakverk av högsta kvalitet och av noga utvalda råvaror. Företaget behöver en webbplats för att potentiella kunder ska kunna hitta information om bageriet, se sortimentet, öppettider och kontaktuppgifter.

Syftet med projektet är att skapa en inbjudande webbplats som ska visa upp sortimentet och locka kunder att vilja besöka bageriet.

Målgruppen är vuxna personer som bor i området runt ”Bakarstaden”.

Planen är att presentera företaget på en ”Om oss”-sida. Presentera sortimentet på en ”Sortiment”-sida genom att läsa in artiklar från en extern fil. Detta för att underlätta ändringar i sortimentet. Samt en ”Kontakt”-sida med ett formulär för att kontakta företaget.

Detta är det som jag absolut vill få med i projektet. Om det finns mer tid när jag är klar, hoppas jag kunna lägga till fler delar.

Tidsramen för projektet är två veckor.

1 Teori

För att förstå beskrivningen av utvecklingen av webbplatsen följer här en förklaring av några centrala begrepp.

Wireframe är en ritning av hur webbplatsen ska vara uppbyggd. Som ett skelett utan design [1].

Mockup är en visuell representation av den färdiga produkten. Den visar utseendet på elementen, tex knappar, färger, bilder, ikoner och typografi [1].

Versionshantering är ett sätt att spara ögonblicksbilder av arbetet under tiden som utvecklingen sker. På så vis finns hela tiden progressionen sparad och man kan lätt återställa filer till ett tidigare läge, se när och vad som ändrats och av vem [2].

HTML står för HyperText Markup Language och det är det kodspråk som används för att skapa webbsidor. Med HTML bygger man webbsidans struktur och innehåll. HTML består av olika element, tex rubriker, paragrafer och länkar **Fel! Hittar inte referenskälla.**.

CSS står för Cascading Style Sheets och är det språk som används för att styra utseendet på en webbsida. CSS-koden beskriver hur HTML-elementen ska se ut och flera webbsidor kan stylas samtidigt genom att länka till externa CSS-filer [4].

Responsiv webbdesign innebär att webbsidans utseende anpassar sig för att se bra ut på alla sorters skärmar [5].

CSS Media Queries låter oss ange utseende beroende på vilken enhet användaren har. Man kan tex ställa in regler som gäller skärmar med vissa bredder [6].

CSS Flexbox står för CSS Flexible Box Layout module och är en layoutmodell för att arrangera innehåll antingen horisontellt eller vertikalt på ett responsivt sätt [7].

CSS Grid, CSS Grid Layout Module är ett grid-baserat layoutsystem med både rader och kolumner som gör det lätt att skapa avancerad responsiv layout [8].

JavaScript är webbens programmeringsspråk och det kan ändra både HTML och CSS, samt räkna, ändra och validera data [9][9].

Händelsehantering, en händelse kan tex vara en knapp som klickas, att musen förs över ett element eller en tangent som trycks ner. JavaScript kan reagera på händelser och alltså få något att hända. För att lyssna på händelser kan man lägga till händelselyssnare [13].

Asynkron programmering låter programmet utföra uppgifter i bakgrunden utan att resten av programmet behöver vänta på att uppgiften blir klar. Med async/await kan man vänta på ett en asynkron operation utförs och sedan gå vidare till nästa instruktion [14].

Fetch API är ett gränssnitt som låter JavaScript ladda ner data från en webbserver asynkront. Metoden fetch() returnerar ett löfte (promise) som resulterar i ett svar (response) [15].

JSON står för JavaScript Object Notation och är ett textformat för att lagra och utbyta data [16].

2 Metod

För att underlätta utvecklingen kommer arbetet att inleds med att webbplatsens struktur, innehåll och navigering planeras genom att skapa wireframes i designverktyget Figma. Det kommer att vara minst en wireframe för mobil och en för stor skärm.

När arbetet med wireframes är klart kommer jag gå vidare med att planera webbplatsens utseende i en eller flera mockups. Även detta kommer att göras i Figma. Här kommer det framgå vilka färger, typsnitt och bilder som jag planerar att använda.

När innehåll och utseende är planerat kan arbetet med att skapa webbplatsen börja och det blir då lättare att få en bra överblick. Projektet kommer att skapas i Visual Studio Code och versionshanteras genom Github dels för att spara säkra kopior av arbetet och även för att kunna följa arbetets utveckling.

Versionshanteringen kommer att börja i en huvudgren (main) i projektets Github-repository och när grunden för webbplatsen är klar kommer utvecklingsarbetet att flyttas till en utvecklingsgren (dev). Eftersom när delar av utvecklingen blir klar kommer det färdiga arbetet att sparas in på huvudgrenen. Detta är ett bra arbetssätt inför framtiden ifall man har ett projekt som redan är publicerat så att man kan utveckla nya delar separat eller om man är flera som arbetar på samma projekt.

Webbplatsens innehåll kommer att kodas i HTML5 med huvudsakligen semantiska element enligt standard för bra tillgänglighet.

Hur webbplatsen ska se ut kommer att styras genom en extern CSS-fil. Där kommer all design att finnas, ex typsnitt, färger, textstorlekar och bakgrunder. Layouten kommer att styras med CSS Flexbox där element behöver positioneras antingen vågrätt eller lodrätt och CSS Grid där både vågrät och lodrät positionering behövs.

Webbplatsen kommer att utvecklas enligt strategin ”mobile first”, vilket innebär att jag börjar att utveckla för en liten skärm och sedan anpassar designen efter större skärmstorlekar med hjälp av media querys.

Användarinteraktion kommer att implementeras genom JavaScript. Tex med en mobil hamburgermeny och formulärvalidering. För att underlätta underhåll av sortimentet kommer detta att läsas in via JavaScript från en extern fil.

Den färdiga webbplatsen kommer testas i olika webbläsare och både på desktop, surfplatta och mobil. Koden ska också validera korrekt i W3Cs valideringsverktyg för HTML- och CSS-kod.

JavaScript-koden kommer att testas genom att använda webbplatsen med webbläsarens konsollogg framme så att jag kan se om några felmeddelanden dyker upp.

Slutligen kommer webbplatsen att publiceras via Github på Netlify.

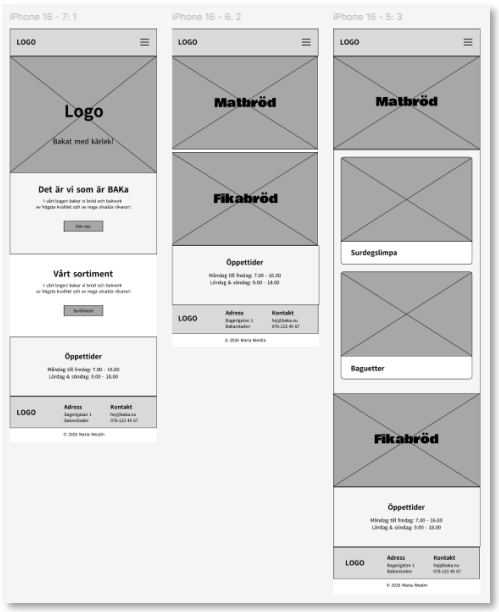
3 Konstruktion

3.1 Planering

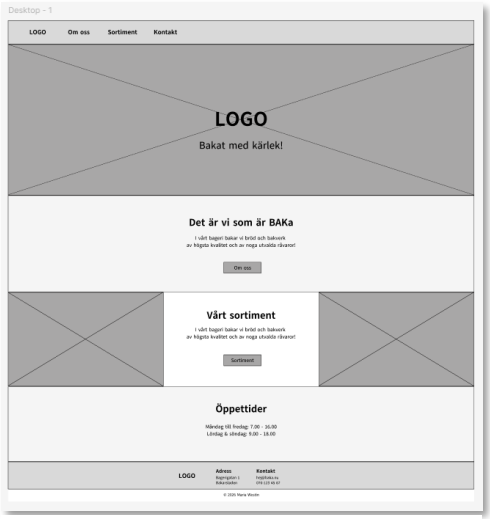
Enligt min plan började jag arbetet i designverktyget Figma. Först skapade jag en wireframe för startsidan i mobilstorlek. Jag valde också att göra en wireframe för sortiment-sidan då jag hade lite olika funderingar kring hur de olika artiklarna skulle kunna presenteras på sidan, se figur 1.

Planen var att göra två bilder som knappar med de två olika artikelgrupperna Matbröd och Fikabröd. Vid klick på en av bilderna skulle artiklarna i den gruppen visas.

Sedan gick jag vidare och gjorde en wireframe för desktop, se figur 2.



Figur 1 – Wireframes mobil



Figur 2 – Wireframe desktop

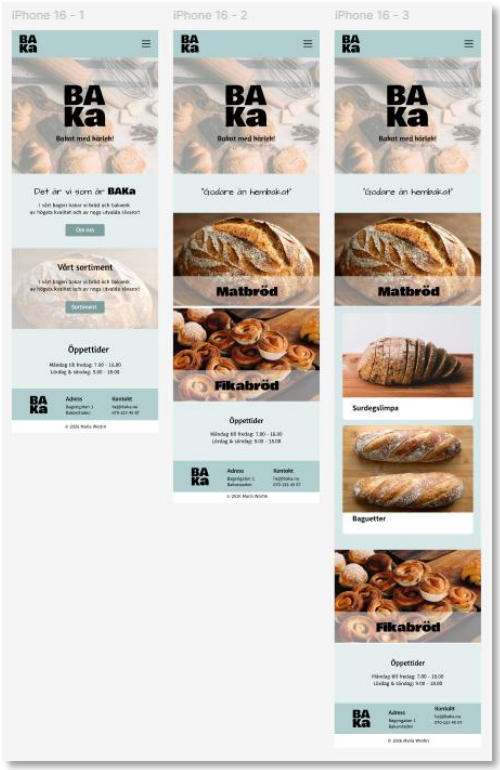
När strukturen var planerad i mina wireframes skulle jag planera utseendet genom att göra mockups i Figma. Men innan jag kunde börja med det behövde jag en logotyp för mitt fiktiva företag.

Logotypen designade jag i Photopea. Under arbetet med logotypen hittade jag också en grundfärg som jag ville använda i den grafiska profilen, se figur 3.

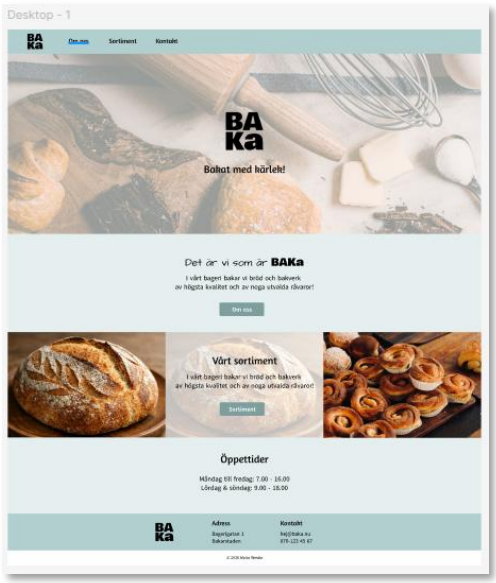


Figur 3 - Logga

När logotyp och grundfärg var klart satte jag i gång med att utveckla den grafiska profilen i Figma. Vilket resulterade i ett antal mockups, se figur 4 och 5.



Figur 4 – Mockups mobil



Figur 5 – Mockup desktop

3.2 HTML & CSS

När planeringen var klar startade jag upp ett nytt projekt i kodeditorn Visual Studio Code och direkt när första filerna var skapade drog jag i gång ett nytt repository i Github för att komma i gång med versionshanteringen.

Jag började arbeta med startsidans element och sedan lade jag till css. Jag valde att positionera menyn med CSS Flexbox för att få länkarna i menyn att flöda horisontellt på större skärm. För att få till denna responsivitet använde jag media querys som lägger till flexbox vid skärmstorlek på minst 750 pixlar.

Bakgrundsbilden på min hero-sektion skulle ha en nedtonad färg och för att inte även logga och text skulle bli genomskinliga, vilket var resultatet när jag satte genomskinlighet (opacity) löste jag det först genom att lägga en genomskinlig ljusgrå div-platta på bakgrunden. Sedan hittade jag en ännu bättre lösning från Radu [10]. Genom att lägga både bakgrundsbild och bakgrundsfärg med RGB-genomskinlighet på elementet och sedan blanda dessa med *background-blend-mode: lighten;* så behövdes inte längre en extra div-platta, se figur 6 och 7.

```html
<main>
    <!-- Sektion 1: Hero -->
    <section class="hero">
        <!-- <div class="transparent"> Behövdes inte längre! -->
            <img src="images/logo.png" alt="Baka Logo" class="hero-logo">
            <h1>Bakat med kärlek!</h1>
        <!-- </div> -->
    </section>
```

Figur 6 – HTML hero

```css
.hero {
    background-image: url("/images/1-hero.jpg");
    background-color: rgba(255, 255, 255, 0.7);
    background-blend-mode: lighten;
    background-size: cover;
    background-position: center;
    padding: 10%;
    text-align: center;
}

/* Behövdes inte längre
.transparent {
    background-color: rgba(255, 255, 255, 0.6);
    padding: 10%;
} */
```

Figur 7 – CSS hero

Sektionen sortiment på startsidan ligger också på en genomskinlig bakgrund så det löste jag lika som på hero-sektionen. Sedan ville jag att sortiment-sektionen skulle ändra utseende två gånger när skärmen blev större. Det löste jag genom att lägga två tomma element, ett före och ett efter. Med en media query på minsta skärmstorlek 750 pixlar, lade jag till innehåll i det första tomma elementet och med flexbox gjorde detta att jag fick två element bredvid varandra. Sortiment-sektionen till höger och en bild till vänster. Sedan lade jag till en till media query vid minsta skärmstorlek 1000 pixlar och där läggs också innehåll i det andra tomma elementet till och resultatet blev tre element bredvid varandra. Sortiment-sektionen i mitten och en bild på varje sida.

Sidfoten är också den positionerad med flexbox eftersom jag vill rada upp elementen horisontellt.

På sidan ”Om oss” valde jag att använda CSS Grid för att placera text och bild i både rader och kolumner. Först i mobilstorlek ligger elementen i en kolumn: bild 1, text 1, bild 2 och text 2. Sedan använder jag media query för att ändra till två kolumner vid större skärmbredd. Men jag positionerar också elementen efter grid-linjerna för att flytta bild två till höger och text två till vänster, se figur 8. Denna lösning hittade jag på MDN [11] i ett senare skede. Först hade jag lagt till en extra bild som låg nedanför i HTML-koden och därför naturligt placerades till höger. Sedan visade/döljde jag bilderna i en media query.

```css
/* 2 rader x 2 kolumner */
.grid-wrap-about {
    grid-template-columns: 1fr 1fr;
    grid-template-rows: 1fr 1fr;
}

#about-img1 {
    aspect-ratio: auto;
}
/* Placerar texten till vänster i andra raden  */
#about-text2 {
    grid-column: 1/2;
    grid-row: 2/3;
}
/* Placerar bilden till höger */
#about-img2 {
    grid-column: 2/3;
    aspect-ratio: auto;
}
```

Figur 8 – CSS ”Om oss”

Även på sortiment-sidan har jag använt CSS Grid för att positionera elementen. Här har jag gjort en grid med sex kolumner och använder ”span” för att specificera hur breda de olika elementen ska vara.

3.3 JavaScript

Hamburgermenyn som syns på liten skärm får sin funktionalitet genom JavaScript. Jag har tagit hjälp av den instruktionsfilm för hamburgermeny som finns i kursmaterialet [12]. Jag använder en händelselyssnare som vid klick på hamburgerikonen kör en funktion som använder metoden toggle() för att växla mellan att lägga till eller ta bort CSS-klassen ”open” på menyn, se figur 9. På så vis syns menyn när man klickat på ikonen och döljs igen om man klickar igen.

```
// Hamburgermeny
// Hämta elementen från DOM
const hamburgerBtn = document.querySelector(".menu-toggle");
const menu = document.querySelector(".menu");

hamburgerBtn.addEventListener("click", function(){
    menu.classList.toggle("open")});
```

Figur 9 – JavaScript meny

Nästa sida där det finns JavaScript är på sortimentsidan. Detta skript ligger i en egen js-fil för att hålla koden separerad och lättläst. Här har jag valt att lägga artiklarna i två olika json-filer, en för varje kategori, och sedan använda JavaScript för att hämta in datan från filerna i asynkrona funktioner där jag använder await fetch. Funktionen körs när användaren klickar på någon av bilderna ”Matbröd” eller ”Fikabröd” och skriver ut artiklarna från vald kategori till sidan.

Formuläret på kontaktsidan valideras också med JavaScript. Även här ligger detta skript i en egen js-fil. Valideringen sker när skicka knappen klickas genom en händelselyssnare. Formulärfälten kontrolleras genom if-satser som genererar fel ifall fälten är tomma. Felen loggas i en lista (array) som sedan skrivs ut som felmeddelanden på sidan. När alla fält är ifyllda kan formuläret skickas. Jag har inte lagt till funktionalitet för att ta hand om den skickade informationen men ett meddelande skrivs ut på sidan om att meddelandet har skickats.

4 Resultat

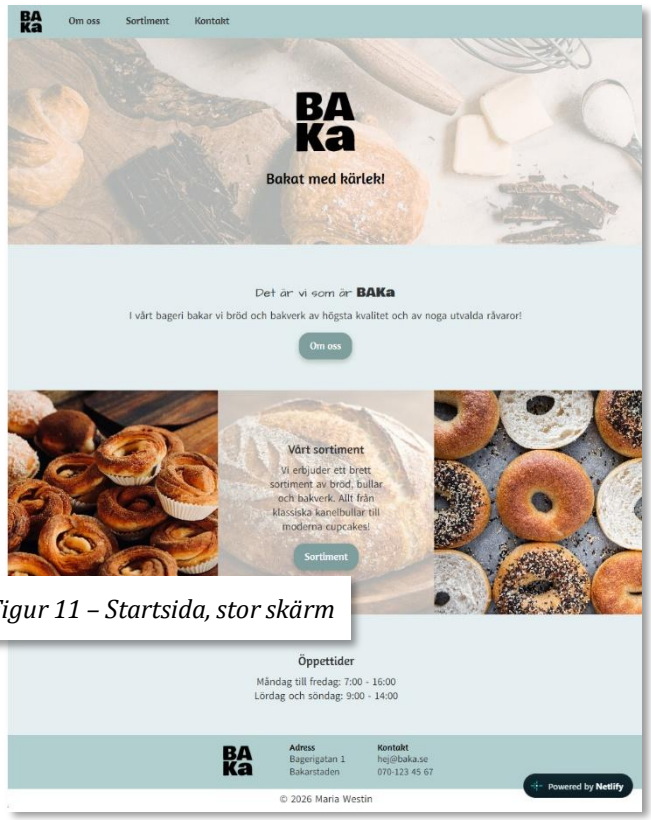
Den färdiga webbplatsen innehåller totalt fyra sidor. En startsida med kort information och länkar till undersidor, se figurerna 9-11. En sida som heter ”Om oss” där bageriet presenteras lite mer utförligt. En sortiment-sida där sortimentet presenteras och en kontakt-sida med ett kontaktformulär.

Längst upp i sidhuvudet finns en meny som på minsta skärmstorlek döljs så att bara en ikon med tre streck syns, se figur 9. När man klickar på ikonen fälls menyn ut, se figur 10, och om man klickar igen döljs den igen. Om skärmen är större ligger länkarna i menyn längst upp på en rad hela tiden, se figur 11.

Startsidans sortiment-sektion har tre olika utseenden beroende på skärmens storlek. På de minsta skärmarna består den av ett textblock på en bakgrundsbild se figur 9. På en skärmstorlek på minst 750 pixlar tillkommer en bild till vänster, se figur 12, och på en skärmstorlek på minst 1000 pixlar tillkommer ytterligare en bild till höger, se figur 11.



Figur 9 - Startsida



Figur 11 – Startsida, stor skärm



Figur 10 – Utfälld meny

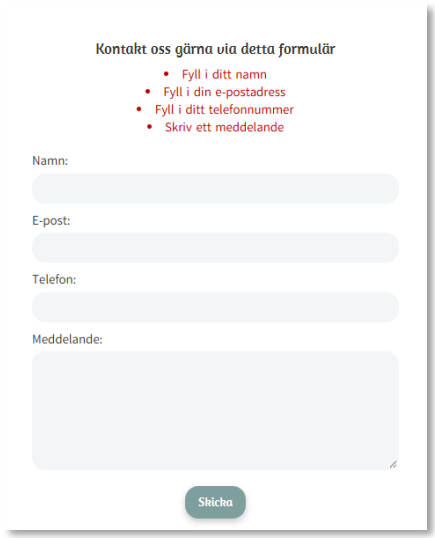


Figur 12 – Mellanstor skärm

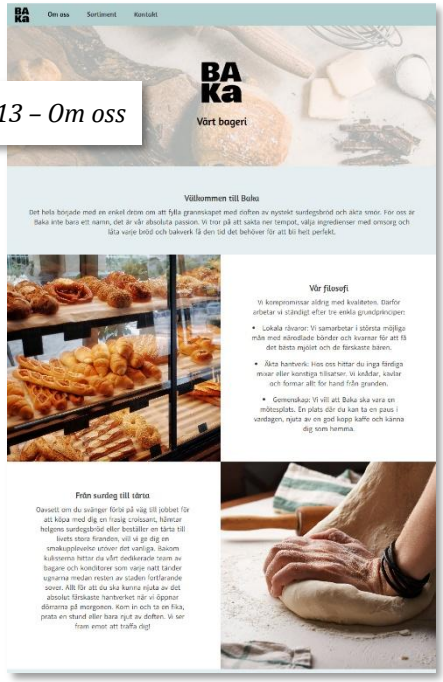
På sidan ”Om oss” ligger innehållet i en kolumn på små skärmar men på stor skärm ligger innehållet i två kolumner så att det bildar rutor, se figur 13.

Sortiment-sidan visar först två kategorier: ”Matbröd” och ”Fikabröd”. När man klickar på en kategori så visas artiklarna i den valda kategorin nedanför, se figur 14. På liten skärm visas matbrödsartiklarna under bilden matbröd och fikabrödsartiklarna visas under bilden fikabröd.

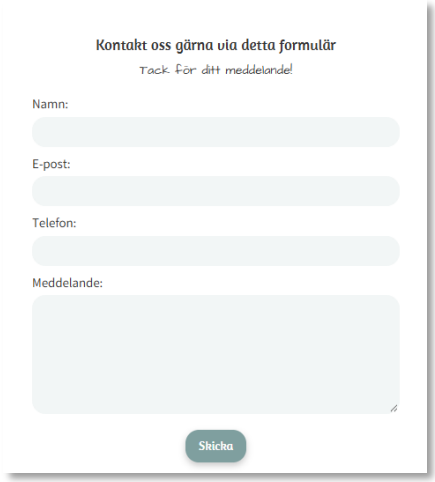
På kontakt-sidan finns ett kontaktformulär som visar felmeddelanden om inte alla fält är ifyllda, se figur 15. Det visas också ett meddelande om att formuläret är skickat om fälten är ifyllda när skicka knappen klickas, se figur 16.



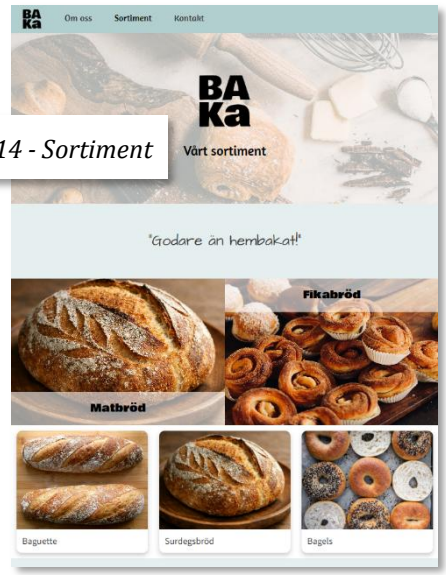
Figur 15 – Felmeddelanden i kontaktformulär



Figur 13 – Om oss



Figur 16 – Meddelande vid skickat formulär



Figur 14 - Sortiment

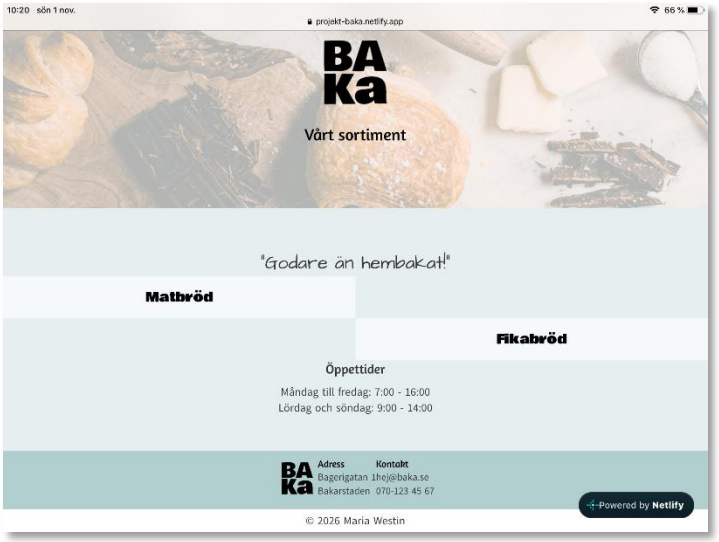
Hela projektet är versionshanterat i GitHub. Jag har använt två grenar (branches). En huvudgren (main) där jag började jobbet. Sedan flyttade jag utvecklingen till en utvecklingsgren (dev). Vid några tillfällen flyttade jag över den färdigutvecklade koden (merge) från utvecklingsgrenen till huvudgrenen för att sedan fortsätta utveckla något nytt i utvecklingsgrenen.

4.1 Test

Jag har testat den färdiga webbplatsen på min mobiltelefon (Samsung Galaxy Note20 Ultra) och alla länkar och funktioner fungerar som de ska. Tyvärr täcker ”Powered by Netlify” en del av sidfoten. Detta är en dekal som läggs till automatiskt när sidan publicerats på Netlify. Det är också lite otydligt att bilderna ”Matbröd” och ”Fikabröd” går att klicka på när man inte har en mus som ändrar pekare när man för den över.

På desktop har jag testat webbplatsen i Google Chrome, Microsoft Edge och Firefox utan några problem. Även på Lenovo Idea Tab fungerar allt som det ska.

Vi har också en gammal iPad Air hemma, som kör iOS 12.5.8. När jag testade i Safari på den så var det två saker som inte fungerade. Bakgrundsbilderna på kategorierna ”Matbröd” och ”Fikabröd” visas inte och avståndet mellan elementen i sidfoten försvann också.



Figur 17 – Test på iPad Air

HTML-sidorna och CSS-filen validerar utan fel eller varningar i W3C-validatorerna.

Jag har också testat sidorna med konsolen framme för att se om det dyker upp några JavaScript fel, men allt verkar fungera.

5 Slutsatser

Enligt min uppfattning uppfyller webbplatsen de behov och mål som tidigare definierats. Det är en inbjudande webbplats som presenterar information om bageriet, öppettider och kontaktuppgifter.

Artiklarna hämtas från en extern fil som planerat och detta gör att det skulle gå att uppdatera sortimentet endast genom att uppdatera filen.

Även testningen tycker jag är godkänd. Att det fanns saker som inte fungerade på den gamla iPaden tycker jag är överkomligt. Men det är något som jag skulle kunna förbättra om jag arbetar vidare med projektet. Jag skulle också gärna ha lagt till någon form av pil på kategorierna ”Matbröd” och ”Fikabröd” på sortimentsidan för att visa att dessa är klickbara när man besöker webbplatsen på mobil eller surfplatta.

Versionshanteringen i GitHub gick bra. Efter att jag hade utvecklat hamburgermenyn i utvecklingsgrenen och överfört det färdiga jobbet till huvudgrenen för första gången, glömde jag att återgå till utvecklingsgrenen för att fortsätta utveckla. Jag gjorde då några ändringar i huvudgrenen och upptäckte sedan att jag skulle byta gren och gjorde då det utan att först göra en merge. Det uppstod då lite förvirring då ändringarna jag hade gjort saknades i utvecklingsgrenen. Men jag fick till det och sedan kände jag att jag hade lite bättre förståelse för att arbeta i olika grenar och det flöt på bra.

En sak jag kämpade med ett tag var att få till bilderna på ”Om oss”-sidan så att de skulle fylla ut höjden på textblocket men inte skapa tomrum under textblocket när skärmen blev större. Detta löste jag till slut med att byta från flex till grid och lägga till aspect-ratio.

Jag känner att jag lärt mig flera nya saker angående CSS Grid under det här projektet. Vid flera tillfällen började jag med onödigt krångliga lösningar med flera likadana element som fick döljas/visas vid olika skärmstorlekar och även dubbla JavaScript funktioner för att skriva ut innehåll i olika HTML-element beroende på skärmstorlek. Men när jag förstod mig på hur man kunde positionera elementen i grid efter grid-linjer och även span så blev lösningarna mycket enklare.

En annan sak som skapade förvirring ett tag var varför inte formulärknappen fungerade när jag jobbade med JavaScript valideringen. Men det visade sig att jag hade lagt knappen utanför formuläret.

Hur jag skulle göra med formuläret när det skulle skickas var också en fråga som dök upp. Här hade jag önskat en funktionalitet för att ta hand om den skickade informationen, men jag valde att bara skriva ut att formuläret är skickat.

Det fanns tyvärr inte tid till ytterligare delar som jag hade hoppats, men det som jag absolut ville ha med är med.

Källförteckning

[1] Figma, ”Wireframe vs. mock-up: what’s the difference?”, https://www.figma.com/resource-library/wireframe-vs-mockup/ Hämtad 2026-11-01.

[2] Git, ”Kom igång – Om versionshantering”, https://git-scm.com/book/sv/v2/Kom-ig%C3%A5ng-Om-versionshantering Hämtad 2026-11-01.

[3] W3 Schools, ”HTML Introduction”, https://www.w3schools.com/html/html_intro.asp Hämtad 2026-11-01.

[4] W3 Schools, ”CSS Introduction”, https://www.w3schools.com/css/css_intro.asp Hämtad 2026-11-01.

[5] W3 Schools, ”HTML Responsive Web Design”, https://www.w3schools.com/html/html_responsive.asp Hämtad 2026-11-01.

[6] W3 Schools, ” CSS Media Queries”, https://www.w3schools.com/css/css3_mediaqueries.asp Hämtad 2026-11-01.

[7] W3 Schools, ”CSS Flexbox”, https://www.w3schools.com/css/css3_flexbox.asp Hämtad 2026-11-01.

[8] W3 Schools, ”CSS Grid”, https://www.w3schools.com/css/css3_flexbox.asp Hämtad 2026-11-01.

[9] W3 Schools, ”What is JavaScript”, https://www.w3schools.com/whatis/whatis_js.asp Hämtad 2026-11-01.

[10] Radu.link, ”Change Background Image Opacity Without Fading the Content in CSS”, https://radu.link/change-background-image-opacity-css/ Publicerad 2025-07-02. Hämtad 2026-10-28.

[11] MDN ” Basic concepts of grid layout” https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout/Basic_concepts Publicerad 2026-08-27. Hämtad 2026-10-28.

[12] Rama Bito, “Mobilmeny (hamburgermeny) med JavaScript” https://youtu.be/4P2XWqrtrY4 Hämtad 2026-11-01.

[13] W3 Schools, ”JavaScript Events”, https://www.w3schools.com/js/js_events.asp Hämtad 2026-11-01.

[14] Code Labs Academy, ”En nybörjarguide till asynkron JavaScript”, https://codelabsacademy.com/sv/blogg/asynchronous-programming/ Hämtad 2026-11-01.

[15] W3 Schools, ”Asynchronous Fetch API”, https://www.w3schools.com/js/js_async_fetch.asp Hämtad 2026-11-01.

[16] W3 Schools, ”JavaScript JSON”, https://www.w3schools.com/js/js_json.asp Hämtad 2026-11-01.